```
                timestamp,
                lat_e6,
                lon_e6,
                altitude_m,
                speed_cms,
                heading_de2,
                vs_cms,
                source_ip
            FROM flight_latest
            ORDER BY timestamp DESC
            LIMIT ?
            """,
            (limit,),
        )

        predicted = []
        for r in cur.fetchall():
            icao = str(r["icao_address"] or "").strip().upper()
            if not icao or icao == "UNKNOWN":
                continue
            if icao in active_icao_set:
                continue

            flight_key = str(r["flight_key"] or "").strip().upper()
            last_seen_wallclock = self._last_ingest_wallclock.get(flight_key)
            if last_seen_wallclock is None:
                # Skip aircraft not seen in the current session ? prevents data from
                # a previous simulation run leaking into live mode as "predictions".
                continue
            age_sec = (now_dt - last_seen_wallclock).total_seconds()
            if age_sec <= 0 or age_sec > max_prediction_age_sec:
                continue

            lat = r["lat_e6"] / 1000000.0
            lon = r["lon_e6"] / 1000000.0
            altitude_m = int(r["altitude_m"] or 0)
            speed_cms = int(r["speed_cms"] or 0)
            heading_de2 = int(r["heading_de2"] or 0)
            vs_cms = int(r["vs_cms"] or 0)

            reg_pred = self._predict_from_linear_regression(
                flight_key=r["flight_key"],
                age_sec=age_sec,
                fallback_lat=lat,
                fallback_lon=lon,
                fallback_alt_m=altitude_m,
            )
            if reg_pred is not None:
                pred_lat = reg_pred["lat"]
                pred_lon = reg_pred["lon"]
                pred_alt_m = reg_pred["alt_m"]
                speed_cms = reg_pred["speed_cms"]
                heading_de2 = reg_pred["heading_de2"]
                vs_cms = reg_pred["vs_cms"]
            else:
                # Fallback to deterministic kinematics if regression cannot fit.
                if speed_cms <= 0 or heading_de2 <= 0:
                    motion = self._estimate_motion_from_recent_samples(r["flight_key"])
                    if motion is not None:
                        if speed_cms <= 0:
                            speed_cms = motion["speed_cms"]
                        if heading_de2 <= 0:
                            heading_de2 = motion["heading_de2"]
                        if vs_cms == 0:
                            vs_cms = motion["vs_cms"]

                speed_mps = speed_cms / 100.0
                heading_deg = heading_de2 / 100.0
                dist_m = speed_mps * age_sec
                heading_rad = math.radians(heading_deg)

                dlat = (dist_m / 111320.0) * math.cos(heading_rad)
                scale = max(0.2, math.cos(math.radians(lat)))
```